

NOTES

Basic SQL Operations



Basic SQL Operations

In this module, you'll learn about creating and managing tables, basic CRUD (Create, Read, Update, Delete) operations, and fundamental querying techniques.

Creating and Managing Tables

What is a Table in SQL?

A **table** in SQL is the basic structure used to store data in a database. It organises data into rows and columns, similar to a spreadsheet.

- **Rows:** Each row (or record) contains a set of data representing an individual item or transaction.
- **Columns:** Each column (or field) represents a specific attribute of the item, such as a product's price, category, or description.

Example: In an e-commerce database, a table named Products may contain columns like Product_id, Brand_Name, DiscountPrice, and Reviews, where each row represents a specific product.

Why Create Tables?

- **Data Organization:** By storing related information together, like all product details in a Products table, it's easier to maintain and retrieve the data.
- **Efficient Queries:** Properly designed tables help in quickly fetching data, like retrieving all products in the "Electronics" category.

- **Data Integrity:** By defining constraints such as Primary Keys, tables ensure that data is unique and accurate.

Example: In an e-commerce store, if all product information is stored in a single table, we can easily search for products, update prices, and track inventory.

How to Create Tables?

In SQL, tables are created using the CREATE TABLE statement, which defines:

- **Columns:** Each column has a name and a data type (e.g., VARCHAR, INT, DECIMAL).
- **Constraints:** Tables can have constraints like PRIMARY KEY to ensure uniqueness of each record, or FOREIGN KEY to link related tables.

Example: Here's a real-world example of creating a Products table in an e-commerce database

Unset

```
CREATE TABLE Products (
  Product_id INT PRIMARY KEY,      --Unique identifier for each product
  URL VARCHAR(255),                --Product link on the website
  Brand_Name VARCHAR(100),         --Name of the product brand
  Category VARCHAR(50),           --Product category
  Individual_category VARCHAR(50), --Sub-category(e.g., Laptop)
  Category_by_Gender VARCHAR(20),  --Gender-specific category
  Description TEXT,                --Product description
  DiscountPrice DECIMAL(10, 2),    --Discounted price
  OriginalPrice DECIMAL(10, 2),   --Original price before discount
  DiscountOffer VARCHAR(10),       --Discount percentage or offer
  SizeOption VARCHAR(50),          --Size options(e.g.Small,Medium)
  Ratings DECIMAL(3, 2),           --Average customer rating
  Reviews INT                      --Number of customer reviews
);
```

- This query creates a Products table that stores information about each product sold in the e-commerce store.
- Each product has a unique Product_id as the **Primary Key**, ensuring that no two products have the same ID.
- Other columns store information about the product's URL, brand, category, pricing, and customer feedback.

Managing Tables

Once a table is created, we can manage it using commands like ALTER (to modify a table), DROP (to delete a table), and TRUNCATE (to remove all data).

Altering a Table: Basic Query

Unset

```
ALTER TABLE Products ADD Stock INT;
```

Altering a Table: Adding a New Column

You may need to modify an existing table to add a new column. For instance, to track product stock, you can add a new column Stock to the Products table.

Unset

```
ALTER TABLE Products  
ADD COLUMN Stock INT DEFAULT 0;
```

This command changes the Products table by adding a new column named Stock. The Stock column is of type INT, which means it will hold whole numbers (like 0, 1, 2, etc). If no number is specified for the Stock when a new product is added, it will automatically be set to 0.

This helps ensure that every product starts with a stock count of zero unless specified otherwise.

Altering a Table: Adding Multiple Columns

To add multiple columns for example Supplier and ManufactureDate in one statement:

```
Unset
ALTER TABLE Products
ADD COLUMN Supplier VARCHAR(100),
ADD COLUMN ManufactureDate DATE;
```

This command modifies the Products table by adding two new columns: Supplier and ManufactureDate. This column is of type VARCHAR(100), meaning it can store text strings up to 100 characters long. It is intended to hold the name of the supplier for each product. This column is of type DATE, which will store date values indicating when each product was manufactured.

Altering a Table: Modifying an Existing Column's Data Type

To change the data type of an existing column for example, change DiscountPrice to have a higher precision:

```
Unset
ALTER TABLE Products
MODIFY COLUMN DiscountPrice DECIMAL(12, 2);
```

This command changes the definition of the DiscountPrice column in the Products table. The DiscountPrice column is being set to a data type of DECIMAL(12, 2). This means, the total number of digits that

can be stored is 12 and Out of these 12 digits, 2 digits will be reserved for the decimal part (the cents).

Dropping a Table: Basic DROP TABLE Command

If you no longer need the Products table, you can delete it entirely using the DROP TABLE command.

Unset

```
DROP TABLE Products;
```

This command will remove the entire Products table from the database, along with all its data. The Products table will no longer exist in the database. Be cautious with this operation, as it cannot be undone without a backup.

Dropping a Table: Dropping Multiple Tables at Once

You can specify multiple table names in a single DROP TABLE statement.

Unset

```
DROP TABLE IF EXISTS Products, Orders, Customers;
```

This command deletes three tables—Products, Orders, and Customers—from the database. The phrase IF EXISTS ensures that the command will only attempt to drop each table if it actually exists. This prevents errors that would occur if you tried to drop any of these tables when they are not present in the database.

Example: Let's assume we have already created the Products table and added some sample data for demonstration:

SQL Query to Insert Data:

Unset

```
INSERT INTO Products (Product_id, URL, Brand_Name, Category,
Individual_category, Category_by_Gender, Description, DiscountPrice,
OriginalPrice, DiscountOffer, SizeOption, Ratings, Reviews)
VALUES
(1, 'https://example.com/laptop1', 'Dell', 'Electronics', 'Laptop',
'Unisex', 'High-performance laptop', 899.99, 999.99, '10%', 'N/A',
4.5, 1200),
(2, 'https://example.com/tshirt1', 'Nike', 'Apparel', 'T-Shirt',
'Men', 'Comfortable cotton t-shirt', 19.99, 24.99, '20%', 'S,M,L',
4.3, 800);
```

SQL Query to Retrieve Data:

```
SELECT Product_id, Brand_Name, Category, DiscountPrice, Ratings FROM
Products;
```

Output

Product_id	Brand_Name	Category	DiscountPrice	Ratings
1	Dell	Electronics	899.99	4.50
2	Nike	Apparel	19.99	4.30
NULL	NULL	NULL	NULL	NULL

Basic CRUD Operations in SQL

CRUD operations are essential for interacting with databases and manipulating the data stored in tables. CRUD stands for:

- **Create**
- **Read**
- **Update**
- **Delete**

Create (Inserting Data into Tables)

What is an INSERT operation?

The **INSERT INTO** command is used to add new rows of data into a table. Each row corresponds to a new record.

Why Insert Data?

- To populate your tables with meaningful data, such as adding new products to your e-commerce store.
- This data can then be retrieved or manipulated as needed.

How to Insert Data?

You use the **INSERT INTO** command to specify which columns you are populating and the respective values.

Example: Let's insert a new product into the Products table.

Unset

```
CREATE TABLE Products (  
    Product_id INT PRIMARY KEY,  
    URL VARCHAR(255),  
    Brand_Name VARCHAR(100),  
    Category VARCHAR(50),  
    Individual_category VARCHAR(50),  
    Category_by_Gender VARCHAR(20),  
    Description TEXT,  
    DiscountPrice DECIMAL(10, 2),  
    OriginalPrice DECIMAL(10, 2),  
    DiscountOffer VARCHAR(10),  
    SizeOption VARCHAR(50),  
    Ratings DECIMAL(3, 2),  
    Reviews INT  
);  
  
INSERT INTO Products  
(Product_id, URL, Brand_Name, Category, Individual_category,  
DiscountPrice, OriginalPrice, Ratings, Reviews)  
VALUES  
(1, 'https://example.com/prod1', 'BrandX', 'Electronics', 'Laptops',  
40000, 45000, 4.5, 120);
```

- This query adds a product with Product_id = 1 to the Products table.
- The product belongs to the "Electronics" category, has a discount price of ₹40,000, and has 120 reviews with a rating of 4.5.

Read (Selecting Data from Tables)

What is a SELECT operation?

The **SELECT** statement is used to retrieve data from a table. You can choose to fetch specific columns or all columns.

Why Read Data?

- To view and analyse data.
- In an e-commerce context, you might want to see a list of products, their prices, or customer ratings to make decisions.

How to Select Data?

- You can fetch all the columns using **SELECT ***.
- You can fetch specific columns by listing them after **SELECT**.
- You can filter results using conditions in the **WHERE** clause.

Example: Let's fetch the Product_id, Brand_Name, and DiscountPrice for each product.

Unset

```
CREATE TABLE Products (
  Product_id INT PRIMARY KEY,
  URL VARCHAR(255),
  Brand_Name VARCHAR(100),
  Category VARCHAR(50),
  Individual_category VARCHAR(50),
  Category_by_Gender VARCHAR(20),
  Description TEXT,
  DiscountPrice DECIMAL(10, 2),
  OriginalPrice DECIMAL(10, 2),
  DiscountOffer VARCHAR(10),
  SizeOption VARCHAR(50),
  Ratings DECIMAL(3, 2),
  Reviews INT
);

INSERT INTO Products (Product_id, URL, Brand_Name, Category,
Individual_category, Category_by_Gender, Description, DiscountPrice,
OriginalPrice, DiscountOffer, SizeOption, Ratings, Reviews)
VALUES
(1, 'http://example.com/product1', 'BrandX', 'Electronics',
'Laptops', 'Unisex', 'High-performance laptop with 16GB RAM.',
40000.00, 50000.00, '10%', '15 inches', 4.5, 150),
```

```
(2, 'http://example.com/product2', 'BrandY', 'Electronics',
'Smartphones', 'Unisex', 'Latest smartphone with excellent camera.',
30000.00, 40000.00, '25%', '6.1 inches', 4.2, 200),
(3, 'http://example.com/product3', 'BrandZ', 'Electronics',
'Tablets', 'Unisex', 'Lightweight tablet with 10 hours battery
life.', 20000.00, 25000.00, '20%', '10 inches', 4.0, 100);

SELECT Product_id, Brand_Name, DiscountPrice
FROM Products;
```

- This query retrieves three columns (Product_id, Brand_Name, and DiscountPrice) from the Products table.
- Useful for quickly seeing basic product details like brand and price.

Output



The screenshot shows a database application interface with a 'Result Grid' tab selected. The grid displays the output of the SQL query, showing three rows of product data. The columns are Product_id, Brand_Name, and DiscountPrice. The first row shows Product_id 1, BrandX, and DiscountPrice 40000.00. The second row shows Product_id 2, BrandY, and DiscountPrice 30000.00. The third row shows Product_id 3, BrandZ, and DiscountPrice 20000.00. There is also a row with NULL values. The interface includes a 'Filter Rows' field, an 'Edit' button, and an 'Export/Import' button.

Product_id	Brand_Name	DiscountPrice
1	BrandX	40000.00
2	BrandY	30000.00
3	BrandZ	20000.00
NULL	NULL	NULL

Using Conditions (WHERE Clause):

Sometimes, you need to filter the data. For example, you may want to view only products under a certain price or in a specific category.

Example: Let's select all products in the Electronics category with a DiscountPrice of less than ₹50,000.

Unset

```
SELECT * FROM Products
WHERE Category = 'Electronics'
AND DiscountPrice < 50000;
```

This query filters the Products table and returns only the products that belong to the Electronics category and have a DiscountPrice below ₹50,000.

Output

Product_id	URL	Brand_Name	Category	Individual_category	Category_by_Gender	Description	DiscountPrice	OriginalPrice	DiscountOffer	SizeOption	Ratings	Reviews
1	http://example.com/product1	BrandX	Electronics	Laptops	Unisex	High-performance laptop with 16GB RAM.	40000.00	50000.00	10%	15 inches	4.50	150
2	http://example.com/product2	BrandY	Electronics	Smartphones	Unisex	Latest smartphone with excellent camera.	30000.00	40000.00	25%	6.1 inches	4.20	200
3	http://example.com/product3	BrandZ	Electronics	Tablets	Unisex	Lightweight tablet with 10 hours battery life.	20000.00	25000.00	20%	10 inches	4.00	100
4	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Update (Modifying Data in Tables)

What is an UPDATE operation?

The **UPDATE** statement allows you to modify existing records in a table, such as changing the price or stock of a product.

Why Update Data?

- To correct or change existing data.
- For instance, when a product price changes or inventory levels need updating.

How to Update Data?

- Use the **UPDATE** command with the table name.

- Specify which column(s) to update and provide a condition (using WHERE) to target specific rows.

Example: Let's update the DiscountPrice of the product with Product_id = 1 to ₹38,000.

Unset

```
UPDATE Products
SET DiscountPrice = 38000
WHERE Product_id = 1;
```

This query changes the DiscountPrice of the product with Product_id = 1 to ₹38,000. The DiscountPrice for the product with Product_id = 1 is now updated in the table.

Delete (Removing Data from Tables)

What is a DELETE operation?

The **DELETE** statement removes specific rows from a table. You can delete one record, several records, or all records from the table, depending on the condition you apply.

Why Delete Data?

- To remove obsolete, inaccurate, or unwanted data.
- For example, if a product is discontinued, you may want to remove it from the database.

How to Delete Data?

- Use the **DELETE** command, followed by the condition to specify which rows to remove.

Example: Let's delete the product with Product_id = 1.

Unset

```
DELETE FROM Products  
WHERE Product_id = 1;
```

This query removes the product with Product_id = 1 from the Products table. The row representing the product with Product_id = 1 is deleted from the table.

Basic Querying Techniques in SQL

SQL provides powerful tools for retrieving and manipulating data. Here, we'll explore some basic querying techniques used to interact with data in your tables.

Selecting Specific Columns

What is Selecting Specific Columns?

In SQL, you can use the **SELECT** statement to retrieve only the specific columns you need, rather than selecting all columns from a table.

Why Select Specific Columns?

- **Efficiency:** Selecting only the necessary columns reduces the amount of data returned, improving query performance and making the results easier to understand.
- **Readability:** Selecting specific columns makes the results more focused and cleaner, especially when you don't need all the data.

How to Select Specific Columns?

Use the **SELECT** statement followed by the columns you want to retrieve, separated by commas.

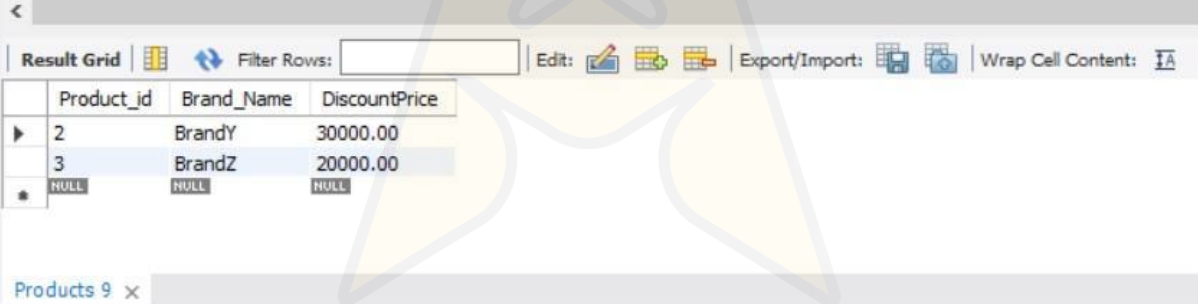
Example: Let's retrieve only the Product_id, Brand_Name, and DiscountPrice for each product.

Unset

```
SELECT Product_id, Brand_Name, DiscountPrice
FROM Products;
```

This query fetches the product ID, brand name, and discounted price from the Products table. It doesn't retrieve other details like the URL or category.

Output



	Product_id	Brand_Name	DiscountPrice
▶	2	Brand_Y	30000.00
	3	Brand_Z	20000.00
*	NULL	NULL	NULL

Products 9 x

Filtering Data with WHERE

What is Filtering?

The **WHERE** clause allows you to apply conditions to your query so that only specific rows (records) are returned.

Why Filter Data?

- **Precision:** It helps you retrieve only the data that meets certain criteria, such as products in a specific category or under a certain price.
- **Data Management:** Filtering can help in isolating particular records for analysis, such as finding all products from a specific brand.

How to Filter Data?

Use the WHERE clause in combination with SELECT to specify conditions on columns.

Example: Let's retrieve all products from the brand BrandX.

Unset

```
SELECT *
FROM Products
WHERE Brand_Name = 'BrandY';
```

This query returns all the columns for products that have the Brand_Name set to '**BrandX**'. The **WHERE** clause filters the results based on the specified condition.

Output

Product_id	URL	Brand_Name	Category	Individual_category	Category_by_Gender	Description	DiscountPrice	OriginalPrice	DiscountOffer	SizeOption	Ratings	Reviews
2	http://example.com/product2	BrandY	Electronics	Smartphones	Unisex	Latest smartphone with excellent camera.	30000.00	40000.00	25%	6.1 inches	4.20	200
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Sorting Data

What is Sorting?

The **ORDER BY** clause is used to arrange the rows in a result set either in **ascending (ASC)** or **descending (DESC)** order, based on one or more columns.

Why Sort Data?

- **Organising Results:** Sorting makes it easier to analyse and visualise the data. For example, sorting products by price allows you to easily see the most expensive or the least expensive items.
- **Improved Insights:** Sorted data provides a more structured view, making comparisons simpler.

How to Sort Data?

Use the **ORDER BY** clause to specify the column by which you want to sort, and optionally, the sorting order (**ASC** for ascending, **DESC** for descending).

Example: Let's retrieve all products sorted by DiscountPrice in descending order, showing the highest prices first.

Unset

```
SELECT *  
FROM Products  
ORDER BY DiscountPrice DESC;
```

This query retrieves all the products and sorts them based on the DiscountPrice column in descending order (from highest to lowest).

Output

Product_id	URL	Brand_Name	Category	Individual_category	Category_by_Gender	Description	DiscountPrice	OriginalPrice	DiscountOffer	SizeOption	Ratings	Reviews
1	http://example.com/product1	BrandX	Electronics	Laptops	Unisex	High-performance laptop with 16GB RAM.	40000.00	50000.00	10%	15 inches	4.50	150
2	http://example.com/product2	BrandY	Electronics	Smartphones	Unisex	Latest smartphone with excellent camera.	30000.00	40000.00	25%	6.1 inches	4.20	200
3	http://example.com/product3	BrandZ	Electronics	Tablets	Unisex	Lightweight tablet with 10 hours battery life.	20000.00	25000.00	20%	10 inches	4.00	100
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Products 11 x

Aggregating Data

What is Aggregation?

Aggregation refers to summarising or calculating data across multiple records using SQL functions like:

- **COUNT():** Counts the number of rows.
- **SUM():** Adds up values in a column.
- **AVG():** Calculates the average value.
- **MAX():** Finds the highest value.
- **MIN():** Finds the lowest value.

COUNT() – Counting Rows

COUNT() is used to count the number of rows in a table or the number of non-null values in a column. It is used to determine how many records meet certain conditions or how many total records exist in the table.

Example: Let's count the total number of products in each category.

Unset

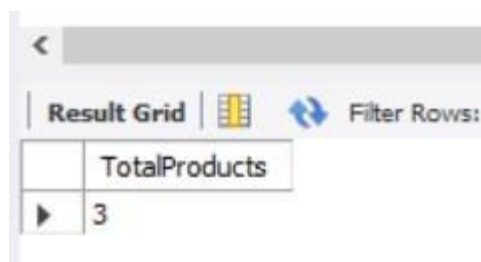
```
CREATE TABLE Products (
  Product_id INT PRIMARY KEY,
  URL VARCHAR(255),
  Brand_Name VARCHAR(100),
  Category VARCHAR(50),
  Individual_category VARCHAR(50),
  Category_by_Gender VARCHAR(20),
  Description TEXT,
  DiscountPrice DECIMAL(10, 2),
  OriginalPrice DECIMAL(10, 2),
  DiscountOffer VARCHAR(10),
  SizeOption VARCHAR(50),
  Ratings DECIMAL(3, 2),
  Reviews INT
);

INSERT INTO Products (Product_id, URL, Brand_Name, Category,
  Individual_category, Category_by_Gender, Description, DiscountPrice,
  OriginalPrice, DiscountOffer, SizeOption, Ratings, Reviews)
VALUES
(1, 'http://example.com/product1', 'BrandX', 'Electronics',
  'Laptops', 'Unisex', 'High-performance laptop with 16GB RAM.',
  40000.00, 50000.00, '10%', '15 inches', 4.5, 150),
(2, 'http://example.com/product2', 'BrandY', 'Electronics',
  'Smartphones', 'Unisex', 'Latest smartphone with excellent camera.',
  30000.00, 40000.00, '25%', '6.1 inches', 4.2, 200),
(3, 'http://example.com/product3', 'BrandZ', 'Electronics',
  'Tablets', 'Unisex', 'Lightweight tablet with 10 hours battery
  life.', 20000.00, 25000.00, '20%', '10 inches', 4.0, 100);

SELECT COUNT(Product_id) AS TotalProducts
FROM Products;
```

The COUNT() function counts how many products exist.

Output



	TotalProducts
▶	3

SUM() – Summing Values

SUM() calculates the total sum of a numeric column. It is used to determine total values such as total sales, total revenue, or total discount applied across products.

Example: Let's calculate the total original price of all products in each category.

Unset

```
SELECT SUM(OriginalPrice) AS TotalRevenue
FROM Products;
```

The SUM() function adds up the OriginalPrice of all products.

Output



	TotalRevenue
▶	115000.00

AVG() – Calculating Averages

AVG() calculates the average value of a numeric column. It is used to calculate averages like the average price of products, average ratings, or average reviews.

Example: Let's calculate the average discount price of products for each category.

Unset

```
SELECT AVG(DiscountPrice) AS AverageDiscountPrice
FROM Products;
```

The **AVG()** function calculates the average of the DiscountPrice for all products.

Output:



Result Grid		Filter Rows:
	AverageDiscountPrice	
▶	30000.000000	

MAX() – Finding Maximum Value

MAX() finds the highest value in a numeric or date column. It is used to identify the maximum price, highest-rated product, or the most reviewed product.

Example: Let's find the highest discount price for products in each category.

Unset

```
SELECT MAX(DiscountPrice) AS MaxDiscountPrice
FROM Products;
```

The **MAX()** function returns the highest DiscountPrice

Output:

```
124 • SELECT MAX(DiscountPrice) AS MaxDiscountPrice
125 FROM Products;
126
127
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
MaxDiscountPrice			
40000.00			

MIN() – Finding Minimum Value

MIN() finds the lowest value in a numeric or date column. It is used to find the minimum price, lowest-rated product, or the product with the fewest reviews.

Example: Let's find the lowest discount price for products in each category.

Unset

```
SELECT MIN(DiscountPrice) AS MinDiscountPrice
FROM Products;
```

The **MIN()** function returns the lowest DiscountPrice.

Output:

```
127 • SELECT MIN(DiscountPrice) AS MinDiscountPrice
128 FROM Products;
129
130
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
MinDiscountPrice			
20000.00			

Note: These aggregate functions are essential tools in SQL for summarising data in your database, helping to gain insights into product prices, sales, and other metrics.

Why Use Aggregates?

Summarise Data: Aggregation functions help generate useful summaries, such as total sales, average ratings, or the number of reviews. By using aggregation, you can quickly summarise data for large datasets, making it easier to spot trends or patterns.

Example: You can calculate the total revenue from sales in different product categories to understand which category performs best.

Gain Insights: Aggregates provide deeper insights into the data by allowing you to calculate important metrics like averages, counts, sums, maximums, and minimums. These insights are critical for making data-driven decisions.

Example: Calculating the average price of products in an e-commerce store helps in understanding pricing trends and adjusting strategies accordingly.

Improve Efficiency: Aggregating data allows you to perform large-scale operations on multiple records in a single query, rather than processing data row by row. This improves the speed and efficiency of data analysis.

Example: Counting the total number of reviews for all products using a single COUNT() function is faster than individually tallying them.

Data Grouping and Comparison: By using the GROUP BY clause, aggregation allows for grouping of data based on columns like category or brand, enabling comparisons between different groups.

Example: You can group products by brand and calculate the total sales for each brand to see which brand is the most popular.

Note: In summary, aggregates are vital for summarising data, providing actionable insights, improving query efficiency, and facilitating easy comparisons across grouped data.



THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)